NexusEnroll Python Codebase — Onboarding Guide

1. What is this project?

NexusEnroll is a proof-of-concept University Course Enrolment System implemented in Python using a **Microservices-style architecture**.

The system covers the three main actors required by the assignment:

- **Student**
- **Faculty**
- **Administrator**

The main purpose of the implementation is not to build a full production web application, but to demonstrate:

- separation of business responsibilities
- object-oriented design
- software design principles
- design patterns
- maintainable and scalable architecture
- core enrolment business rules

The application currently runs as a **console-based proof-of-concept**.

---

2. High-Level Architecture

The code is divided according to business responsibilities.

```
NexusEnroll
│
├── Course Catalogue Service
├── Academic Record Service
├── Enrollment Service
├── Faculty / Grade Service
├── Administrator Service
└── Notification Service
```

Although these services currently run inside the same Python application, they represent logical **microservice boundaries**.

In a production implementation, each one could later become an independent API/service.

---

3. Project Structure

```
NexusEnroll_Python/
│
├── main.py
├── README.md
├── requirements.txt
│
├── nexus/
│   │
│   ├── shared/
│   │   ├── models.py
│   │   ├── exceptions.py
│   │   └── event_bus.py
│   │
│   ├── repositories/
│   │   └── memory.py
│   │
│   ├── course_service/
│   │   └── service.py
│   │
│   ├── academic_service/
│   │   └── service.py
│   │
│   ├── enrollment_service/
│   │   ├── service.py
│   │   ├── validator.py
│   │   ├── strategies.py
│   │   └── commands.py
│   │
│   ├── faculty_service/
│   │   ├── service.py
│   │   └── states.py
│   │
│   ├── admin_service/
│   │   └── service.py
│   │
│   └── notification_service/
│       └── service.py
│
└── tests/
    └── test_nexus_enroll.py
```

---

4. `main.py`

`main.py` is the entry point of the application.

It mainly performs four tasks:

```
Create repositories
        ↓
Create services
        ↓
Connect dependencies
        ↓
Run demonstration scenarios
```

The important functions are:

`build_application()`

Creates all repositories, services, validation strategies and notification listeners.

It acts as the place where the whole system is connected together.

`seed_data()`

Creates sample data such as:

- students
- faculty members
- courses
- prerequisites
- completed courses

Demo functions

The file then demonstrates:

```
Student operations
Faculty operations
Administrator operations
Notifications
Design patterns
```

---

5. Shared Domain Models

Location:

```
nexus/shared/models.py
```

This file contains the core objects used throughout the system.

Main models include:

Student

Represents a university student.

Stores information such as:

```
student ID
name
email
degree programme
account status
```

Faculty

Represents an academic staff member.

Course

Contains:

```
course ID
course name
description
department
instructor
capacity
schedule
prerequisites
```

CourseSchedule

Represents the time and location of a course.

It also contains logic for determining whether two courses overlap.

Enrollment

Represents the relationship between a student and a course.

Possible statuses include:

```
ENROLLED
DROPPED
WAITLISTED
```

GradeRecord

Represents a grade submitted for a student.

The grade also contains its current **State object**.

---

6. Custom Exceptions

Location:

```
nexus/shared/exceptions.py
```

Instead of using generic Python exceptions everywhere, custom exceptions were created.

Examples:

```
PrerequisiteError
CapacityError
TimeConflictError
ValidationError
AuthorizationError
NotFoundError
InvalidStateTransitionError
```

For example, if a student attempts to enrol without completing a prerequisite:

```
Student S003 is missing prerequisites: CS1101
```

This makes errors easier to understand and handle.

---

7. Repository Layer

Location:

```
nexus/repositories/memory.py
```

Repositories act as the data-access layer.

Currently the project uses **in-memory repositories**, meaning the information is stored in Python dictionaries while the application is running.

Examples include:

```
InMemoryCourseRepository
InMemoryStudentRepository
InMemoryFacultyRepository
InMemoryEnrollmentRepository
InMemoryAcademicRecordRepository
InMemoryGradeRepository
```

For example:

```
EnrollmentService
       ↓
EnrollmentRepository
       ↓
Stored enrollment data
```

This means the business services do not have to directly manage dictionaries or databases.

A real implementation could later replace these repositories with:

```
MySQL
PostgreSQL
MongoDB
etc.
```

without redesigning the core business logic.

---

8. Course Catalogue Service

Location:

```
nexus/course_service/service.py
```

Class:

```
CourseCatalogService
```

Responsibilities:

- create courses
- retrieve courses
- search courses
- update course capacity

Students can search using information such as:

```
department
course number
keyword
instructor
```

For example:

```
Computer Science
    ↓
CS1101
CS2303
CS2306
```

---

9. Academic Record Service

Location:

```
nexus/academic_service/service.py
```

Class:

```
AcademicRecordService
```

Responsibilities include:

- storing completed courses
- checking whether a student completed a course
- retrieving academic history
- calculating academic progress

One particularly important operation is:

```
has_completed(student_id, course_id)
```

The Enrollment Service uses this when checking prerequisites.

Example:

```
CS2303
requires
CS1101
```

If the student has completed `CS1101`, the prerequisite validation succeeds.

---

10. Enrollment Service

Location:

```
nexus/enrollment_service/service.py
```

This is one of the central services in the system.

Responsibilities:

- validate enrolment
- enrol a student
- drop a course
- retrieve student schedule
- calculate available seats
- perform administrator override enrolment
- publish enrolment events

A normal enrolment follows approximately:

```
Student requests enrollment
            ↓
Retrieve Student
            ↓
Retrieve Course
            ↓
Run validation strategies
            ↓
Create Enrollment
            ↓
Store Enrollment
            ↓
Publish event
```

```
            ↓
Notification
```

---

11. Strategy Pattern

Location:

```
nexus/enrollment_service/strategies.py
```

The **Strategy Pattern** is used for enrolment validation.

There is a common abstraction:

```
EnrollmentValidationStrategy
```

with different implementations:

```
PrerequisiteValidationStrategy

CapacityValidationStrategy

TimeConflictValidationStrategy
```

The structure is:

```
EnrollmentValidationStrategy
          ▲
          |
  ┌───────┼──────────┐
  |       |          |
Prereq  Capacity   Time Conflict
```

Why Strategy was used

Without Strategy, EnrollmentService could contain something like:

```
check prerequisite
check capacity
check timetable
check future validation rule
```

```
check another rule
...
```

which would become increasingly difficult to maintain.

Instead, each validation rule is independent.

A new rule could later be added:

```
CreditLimitValidationStrategy
```

without rewriting the existing validation system.

---

12. Enrollment Validator

Location:

```
nexus/enrollment_service/validator.py
```

Class:

```
EnrollmentValidator
```

This class receives all validation strategies.

Conceptually:

```
EnrollmentValidator
       │
       ├── Prerequisite Strategy
       ├── Capacity Strategy
       └── Time Conflict Strategy
```

It runs each strategy one after another.

If any strategy fails, enrolment stops.

---

13. Command Pattern

Location:

```
nexus/enrollment_service/commands.py
```

The **Command Pattern** is used for enrolment operations.

Main classes:

```
Command
   ▲
   │
   ├── EnrollCourseCommand
   └── DropCourseCommand
```

Instead of treating enrolment as only a method call, the operation itself becomes an object.

For example:

```
command = EnrollCourseCommand(
    enrollment_service,
    "S001",
    "CS2303"
)

command.execute()
```

Commands also provide an:

```
undo()
```

operation.

This provides a simple demonstration of rollback/compensation behaviour.

It also relates to the transactional requirements of enrolment operations.

---

14. Observer Pattern

Locations:

```
nexus/shared/event_bus.py

nexus/notification_service/service.py
```

The **Observer Pattern** is used for notifications.

The basic flow is:

```
Enrollment Service
       │
       │ publish
       ▼
     EventBus
       │
       │ notify
       ▼
Notification Service
```

For example, Enrollment Service publishes:

```
enrollment.confirmed
```

It does not need to directly know how a notification is sent.

The Notification Service subscribes to that event.

Other events include:

```
enrollment.dropped
course.seat_available
grade.submitted
grade.approved
enrollment.override
```

This keeps notification logic separate from the core business logic.

---

15. Faculty Service

Location:

```
nexus/faculty_service/service.py
```

Responsibilities include:

- view class roster
- create grades
- submit grades
- approve grades

- reject grades
- edit rejected grades
- request course modifications

For example:

```
Faculty
   ↓
Select course
   ↓
View enrolled students
```

The roster contains information such as:

```
Student ID
Student Name
Email
```

---

16. State Pattern

Location:

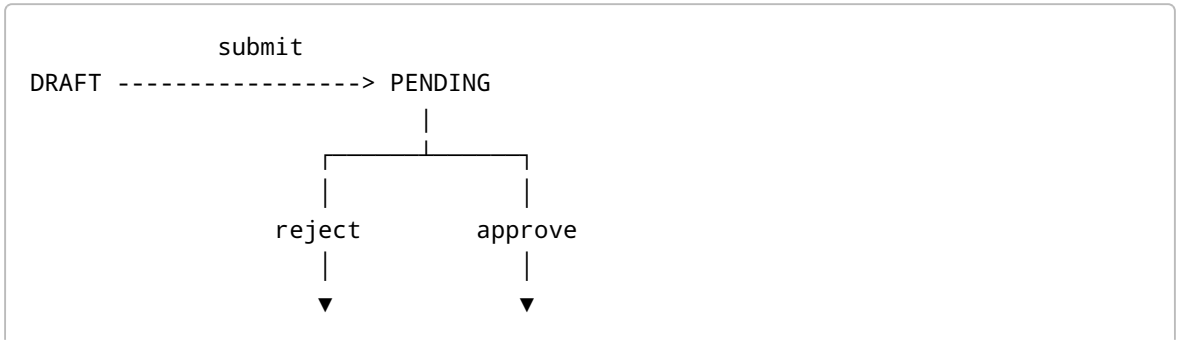
```
nexus/faculty_service/states.py
```

The **State Pattern** controls the lifecycle of grades.

The states currently are:

```
DRAFT
PENDING
SUBMITTED
REJECTED
```

Typical flow:



```
REJECTED      SUBMITTED
    |
  edit
    |
    ▼
  DRAFT
```

Different states determine which operations are allowed.

For example:

- A Draft grade can be submitted.
- A Pending grade can be approved or rejected.
- A Submitted grade cannot simply be submitted again.

This avoids large conditional statements throughout the Faculty Service.

---

17. Administrator Service

Location:

```
nexus/admin_service/service.py
```

Administrator responsibilities include:

User management

```
Add Student
Add Faculty
Deactivate Student
Deactivate Faculty
```

Course management

```
Create Course
Update Course
Delete Course
```

Enrollment management

```
Force-add student
```

The administrator override intentionally bypasses normal validation.

For example:

```
Course capacity = 2

Students currently enrolled = 2

Administrator force-adds another student

Result = 3/2
```

The demonstration therefore shows:

```
150% utilisation
```

This is intentional and demonstrates the administrator override requirement.

Reporting

The administrator service can also generate:

```
Course capacity reports

Faculty workload reports
```

---

18. Notification Service

Location:

```
nexus/notification_service/service.py
```

The Notification Service listens for events generated by other services.

For example:

```
Student enrolled
      ↓
EnrollmentConfirmed Event
      ↓
Notification Service
      ↓
"Student S001 successfully enrolled in CS2303"
```

Currently notifications are printed to the console and stored in a list.

In a future system, the same service could be extended to support:

```
Email
SMS
Push notifications
Mobile application notifications
```

without changing EnrollmentService.

---

19. Tests

Location:

```
tests/test_nexus_enroll.py
```

There are currently **9 automated tests**.

They test:

```
1. Valid enrollment

2. Prerequisite failure

3. Capacity failure

4. Timetable conflict

5. Course drop notification

6. Grade state flow

7. Invalid grade handling

8. Administrator override

9. Capacity reporting
```

The complete test suite currently produces:

```
Ran 9 tests

OK
```

which confirms that the implemented business scenarios work correctly.

---

20. Main Design Patterns Used

Four main design patterns are demonstrated.

Pattern	Location	Purpose
Strategy	Enrollment Service	Different enrolment validation algorithms
Observer	EventBus + Notification Service	Decoupled notifications
State	Faculty/Grade Service	Manage grade lifecycle
Command	Enrollment Service	Encapsulate enrol/drop operations

---

21. Basic Student Scenario

A useful way to understand the whole system is to follow one student enrolment.

Suppose:

```
Student = S001

Course = CS2303
```

The process is:

```
S001 requests CS2303
        ↓
EnrollCourseCommand
        ↓
EnrollmentService
        ↓
Student Repository
        ↓
Course Service
        ↓
EnrollmentValidator
        ↓
```

```
Prerequisite Strategy
        ↓
Capacity Strategy
        ↓
Time Conflict Strategy
        ↓
Enrollment Repository
        ↓
Enrollment created
        ↓
EventBus
        ↓
Notification Service
```

If every validation succeeds:

```
Enrollment successful
```

If one fails:

```
Enrollment rejected
```

---

22. Example Failed Scenario

Suppose:

```
S003 wants CS2303
```

but:

```
CS2303 requires CS1101
```

and S003 has not completed CS1101.

The flow becomes:

```
Enrollment Request
       ↓
PrerequisiteValidationStrategy
       ↓
AcademicRecordService
       ↓
```

```
CS1101 not completed
       ↓
PrerequisiteError
       ↓
Enrollment rejected
```

The other strategies do not need to know how prerequisite checking works.

---

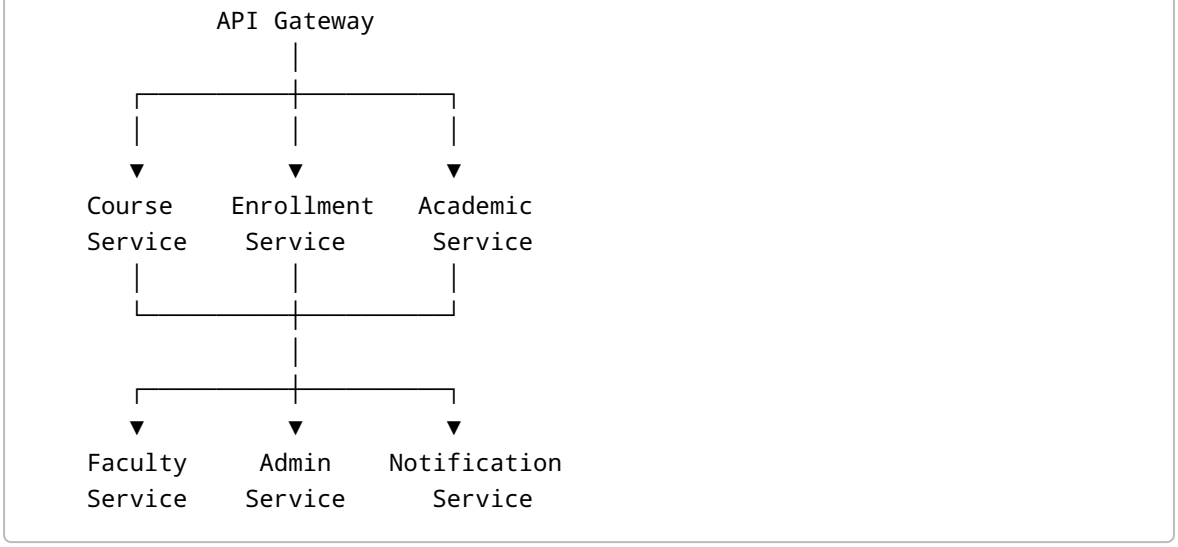
23. How Microservices Fits the Code

Currently these services run together:

```
Course Service
Academic Service
Enrollment Service
Faculty Service
Admin Service
Notification Service
```

This is intentional for the proof-of-concept.

The service boundaries still show how a full system could later be deployed:



A future production version could expose each service using technologies such as REST APIs.

---

24. Important Files to Read First

A new group member should read the code in this order:

```
1. main.py

2. nexus/shared/models.py

3. nexus/repositories/memory.py

4. nexus/course_service/service.py

5. nexus/academic_service/service.py

6. nexus/enrollment_service/strategies.py

7. nexus/enrollment_service/validator.py

8. nexus/enrollment_service/service.py

9. nexus/enrollment_service/commands.py

10. nexus/shared/event_bus.py

11. nexus/notification_service/service.py

12. nexus/faculty_service/states.py

13. nexus/faculty_service/service.py

14. nexus/admin_service/service.py

15. tests/test_nexus_enroll.py
```

Following this order makes the relationships between classes easier to understand.

---

25. How to Run the Project

Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

Then:

```
python main.py
```

The application should demonstrate:

```
Student module
Faculty module
Administrator module
Command/Observer example
Notification log
```

---

26. How to Run the Tests

Run:

```
python -m unittest discover -s tests -v
```

Expected final result:

```
Ran 9 tests

OK
```

---

27. Key Things Every Team Member Should Understand

A team member does **not** need to memorise every line of Python.

However, everyone should understand these concepts:

1. Why the system was divided into business services.
2. What each service is responsible for.
3. How Student, Course and Enrollment relate.
4. How enrolment validation works.
5. Why Strategy Pattern was used for validation.
6. Why Observer Pattern was used for notifications.
7. How State Pattern controls grade submission.
8. Why Command Pattern wraps enrol/drop operations.
9. Why repositories are separate from business services.
10. How `main.py` connects the application.
11. What the automated tests demonstrate.
12. How this proof-of-concept could evolve into independently deployed microservices.

---

28. One-Minute Explanation of the Entire Project

If someone asks, **"What exactly did you build?"**, a simple explanation is:

> NexusEnroll is a Python proof-of-concept university enrolment system designed using a Microservices-style architecture. The system separates responsibilities into Course Catalogue, Enrollment, Academic Record, Faculty, Administrator and Notification services. The Student functionality supports course searching and enrolment with prerequisite, capacity and timetable-conflict validation. Faculty can view rosters and manage grades through a defined grade lifecycle, while administrators can manage users and courses, override enrolment restrictions and generate reports. Four object-oriented design patterns are demonstrated: Strategy for enrolment validation, Observer for notifications, State for the grade lifecycle and Command for enrolment operations. In-memory repositories are used for the proof-of-concept, and nine automated tests verify the main business scenarios.

That is essentially the entire codebase in one explanation.